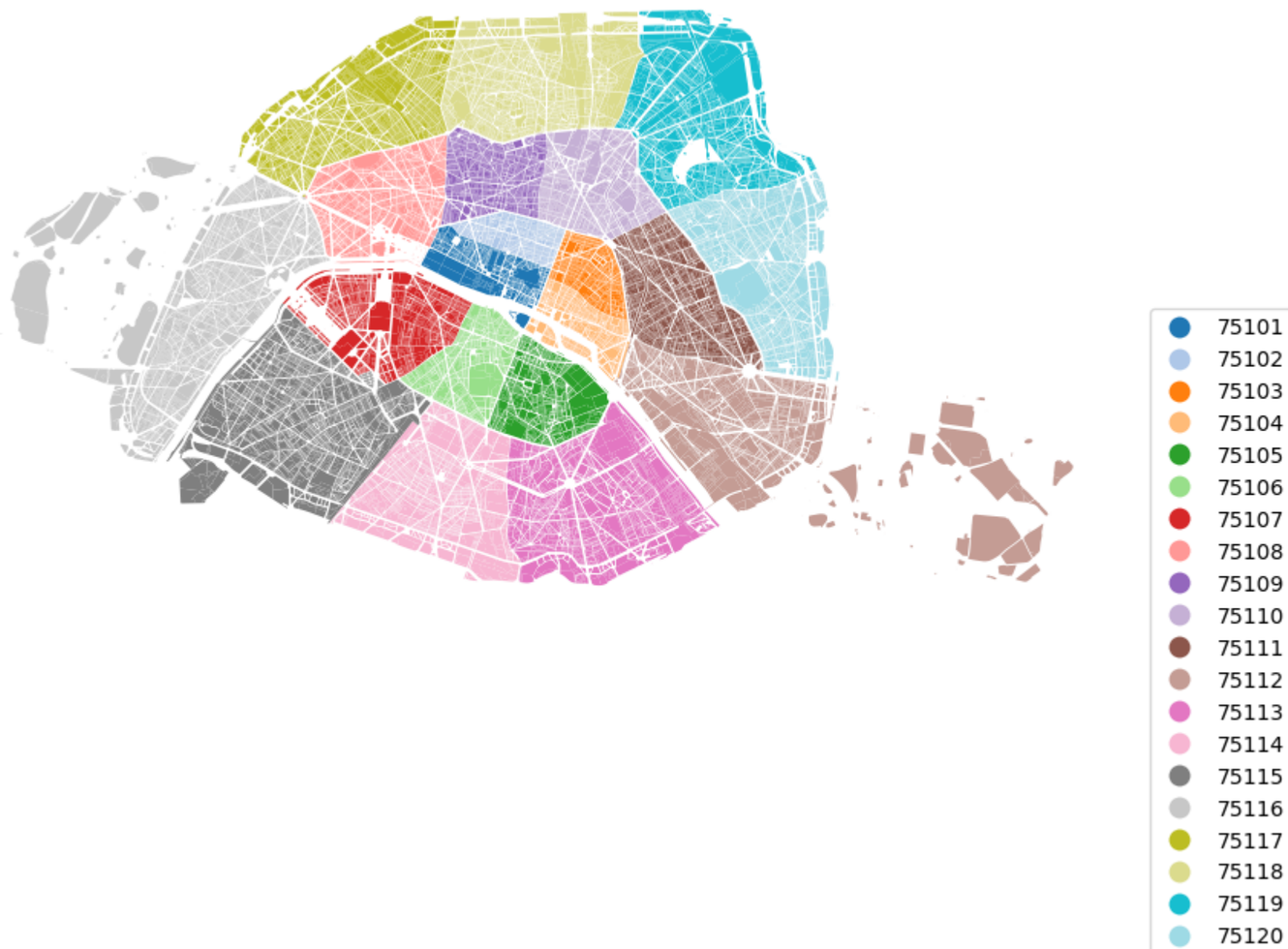
In [ ]:
```python
import pandas as pd
import numpy as np
from numpy.random import seed
import geopandas as gpd
from shapely.geometry import Point
import matplotlib.pyplot as plt
import mapclassify as mc
from shapely.ops import cascaded_union
import seaborn as sns

color = sns.color_palette()
get_ipython().run_line_magic("matplotlib", "inline")
import plotly.offline as py

py.init_notebook_mode(connected=True)
import plotly.express as px

from sklearn.neighbors import KNeighborsRegressor
from sklearn.model_selection import train_test_split, GridSearchCV
import category_encoders as ce
```

In [ ]:
```python
df = pd.read_table("valeursfoncieres-2020.txt", sep="|", low_memory=False)
print("Initial dataframe has the shape:", df.shape)
df.drop_duplicates(keep="last", inplace=True)
print("after dropping duplicates, dataframe has the shape:", df.shape)
df.isnull().mean().mul(100).sort_values(ascending=False)
df = df.loc[:, df.isnull().mean() < 1]
num_col = [
    "Surface Carrez du 1er lot",
    "Surface Carrez du 2eme lot",
    "Surface Carrez du 3eme lot",
    "Surface Carrez du 4eme lot",
    "Surface Carrez du 5eme lot",
    "Valeur fonciere",
]
for col in num_col:
    df[col] = df[col].str.replace(",", ".")
    df[col] = df[col].astype(float)

# filtered the data for only Paris related transaction
df = df.loc[(df["Code postal"] > 74999) & (df["Code postal"] < 75991)]
df["Code postal"] = df["Code postal"].astype(int)

df["Code INSEE"] = df["Code departement"].astype(str) + df["Code commune"].astype(str)
df["id"] = (
    df["Code INSEE"].astype(str)
    + "000"
    + df["Section"]
    + "000"
    + df["No plan"].astype(str)
)

gis_df = gpd.read_file("parcelles.shp")
print("GIS dataframe as the shape:", gis_df.shape)
gis_df.commune = gis_df.commune.astype(int)

# plotting the map of Paris where each district has it's own colour.
```

```
fig, ax = plt.subplots(1, figsize=(10, 10))
gis_df.plot(cmap="tab20", column="commune", ax=ax, categorical=True, legend=True)
leg = ax.get_legend()
leg.set_bbox_to_anchor((1.15, 0.5))
ax.set_axis_off()
plt.show()
```

```
Initial dataframe has the shape: (3484957, 43)
after dropping duplicates, dataframe has the shape: (3346852, 43)
GIS dataframe as the shape: (77720, 9)
```



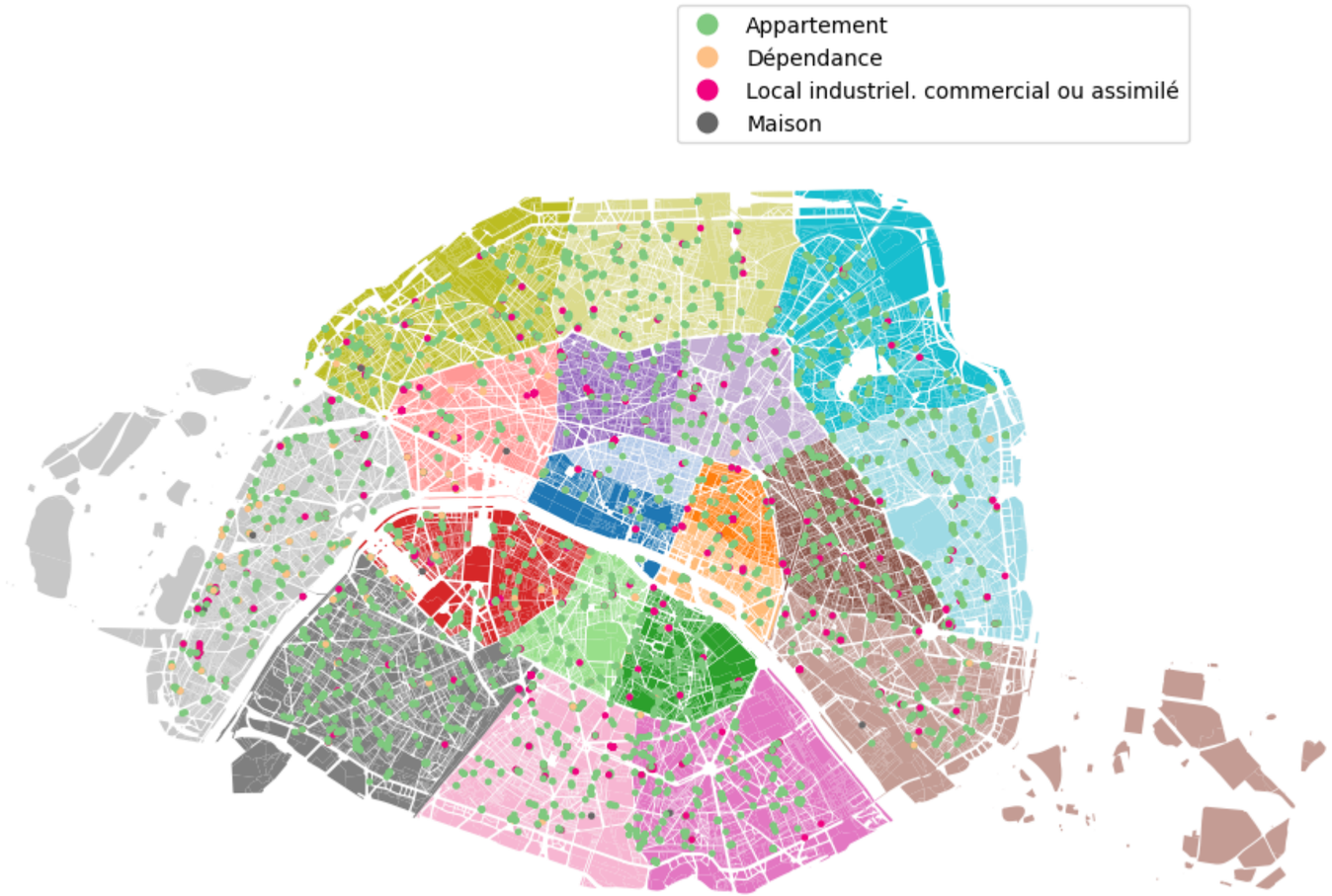
In [ ]:
```
df2 = gis_df.merge(df, how="right", on="id")
df2 = gpd.GeoDataFrame(df2)

df3 = df2.to_crs(2154)
df2["lon"] = df3.centroid.x
df2["lat"] = df3.centroid.y
print("Merged dataframe between gis df and tabular df as the shape:", df2.shape)
gdf = gpd.GeoDataFrame(df2.copy(), geometry=gpd.points_from_xy(df2.lon, df2.lat))
```

```python
# created a
gdf["Total Surface Carrez"] = np.nansum(
    gdf[
        [
            "Surface Carrez du 1er lot",
            "Surface Carrez du 2eme lot",
            "Surface Carrez du 3eme lot",
            "Surface Carrez du 4eme lot",
            "Surface Carrez du 5eme lot",
            "Surface terrain",
            "Surface reelle bati",
        ]
    ],
    axis=1,
)

gdf = gdf.loc[
    (~gdf["Valeur fonciere"].isna())
    & (~gdf.lat.isna())
    & (gdf["Total Surface Carrez"] != 0)
]

gdf.drop(columns=["section", "created", "updated", "numero"], inplace=True)

gdf["price_per_m2"] = gdf["Valeur fonciere"] / gdf["Total Surface Carrez"]

gdf = gdf.loc[:, gdf.isnull().mean() < 1]

# this is dictionary used to check which district a specific coordinate belongs to.
mul_pl_lst = {}
for i in list(gdf.commune.unique()):
    polygons = [gis_df.geometry.loc[gis_df["commune"] == i].unique()]
    mul_pl_lst[i] = gpd.GeoSeries(cascaded_union(polygons[0]))

print("Cleaned dataset as the shape:", gdf.shape)
```

```
Merged dataframe between gis df and tabular df as the shape: (52150, 47)
Cleaned dataset as the shape: (5074, 42)
```

In [ ]:

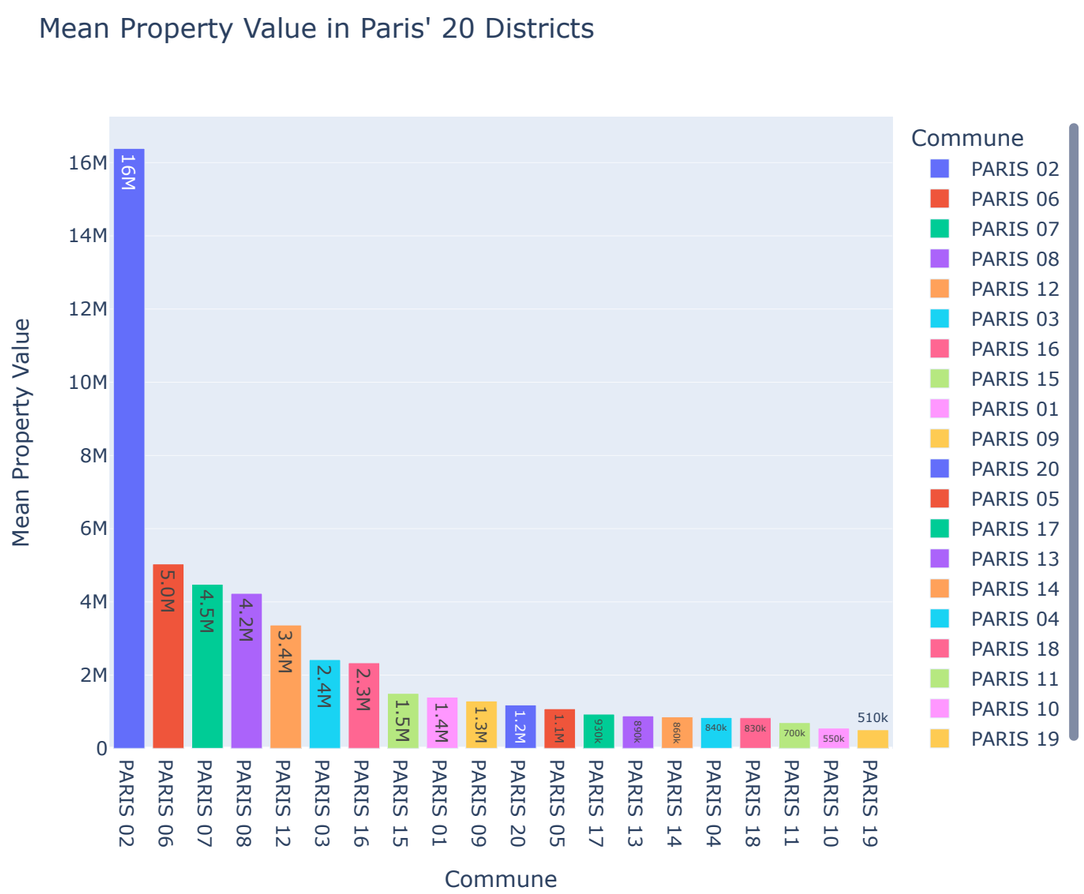
```python
fig, ax = plt.subplots(1, figsize=(12, 12))
gis_df.plot(cmap="tab20", column="commune", ax=ax, categorical=True, legend=True)
gdf.plot(
    ax=ax,
    cmap="Accent",
    column="Type local",
    markersize=5,
    categorical=True,
    legend=True,
)
leg = ax.get_legend()
leg.set_bbox_to_anchor((0.5, 1))
ax.set_axis_off()
plt.show()

# checked to see if the data is spans several years; after filtering and cleaning we have data only from 2020.
gdf["Date mutation"].str[-4:].sort_values().unique()

gdf["Nombre pieces principales"].fillna(1, inplace=True)
```

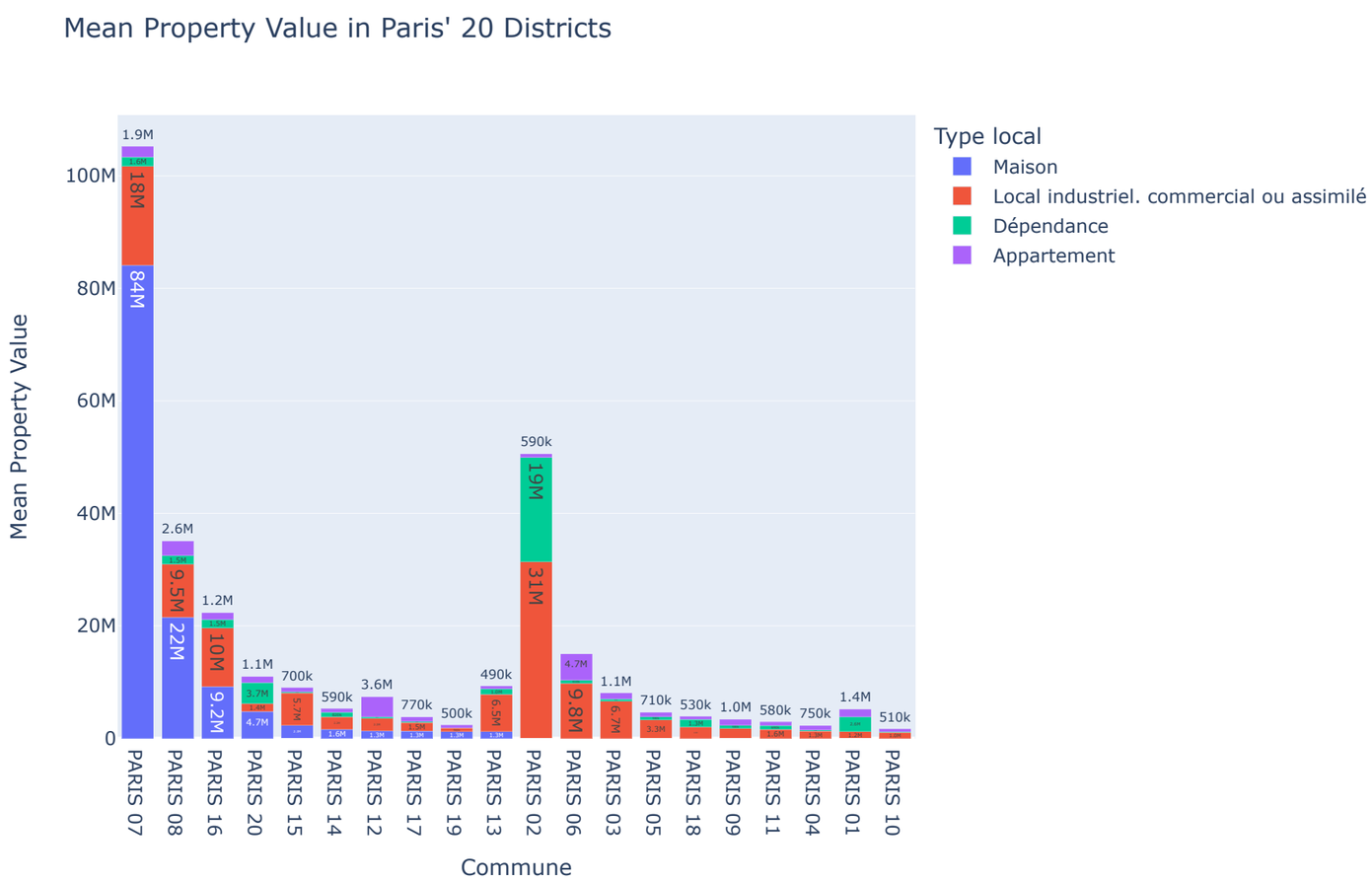


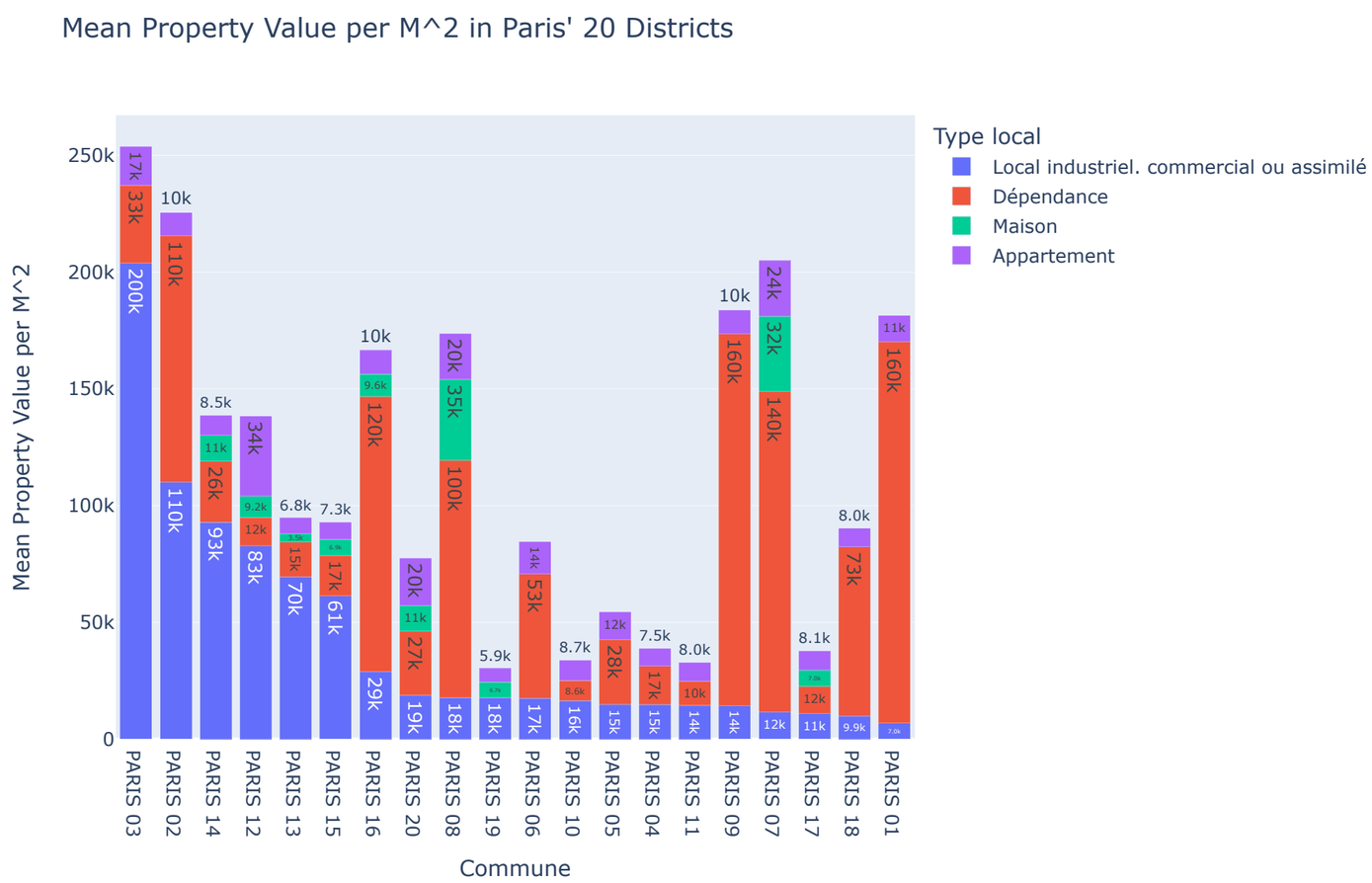
In [ ]:
```
# Found that most of the transaction in our data is Apartment property. On average industrial properties cost more per square meter than Apartments or houses.

# District 15 has the most number of Apartment transaction.
# Majority of the transaction are from district 15 and 16.

tmp_df = (
    gdf.groupby(["Commune"])
    .agg(property_value=("Valeur fonciere", "mean"))
    .sort_values("property_value", ascending=False)
    .reset_index()
)

fig = px.bar(
    tmp_df,
    x="Commune",
    y="property_value",
    title="Mean Property Value in Paris' 20 Districts",
    labels={"property_value": "Mean Property Value"},
    text_auto=".2s",
    color="Commune",
    width=700,
```

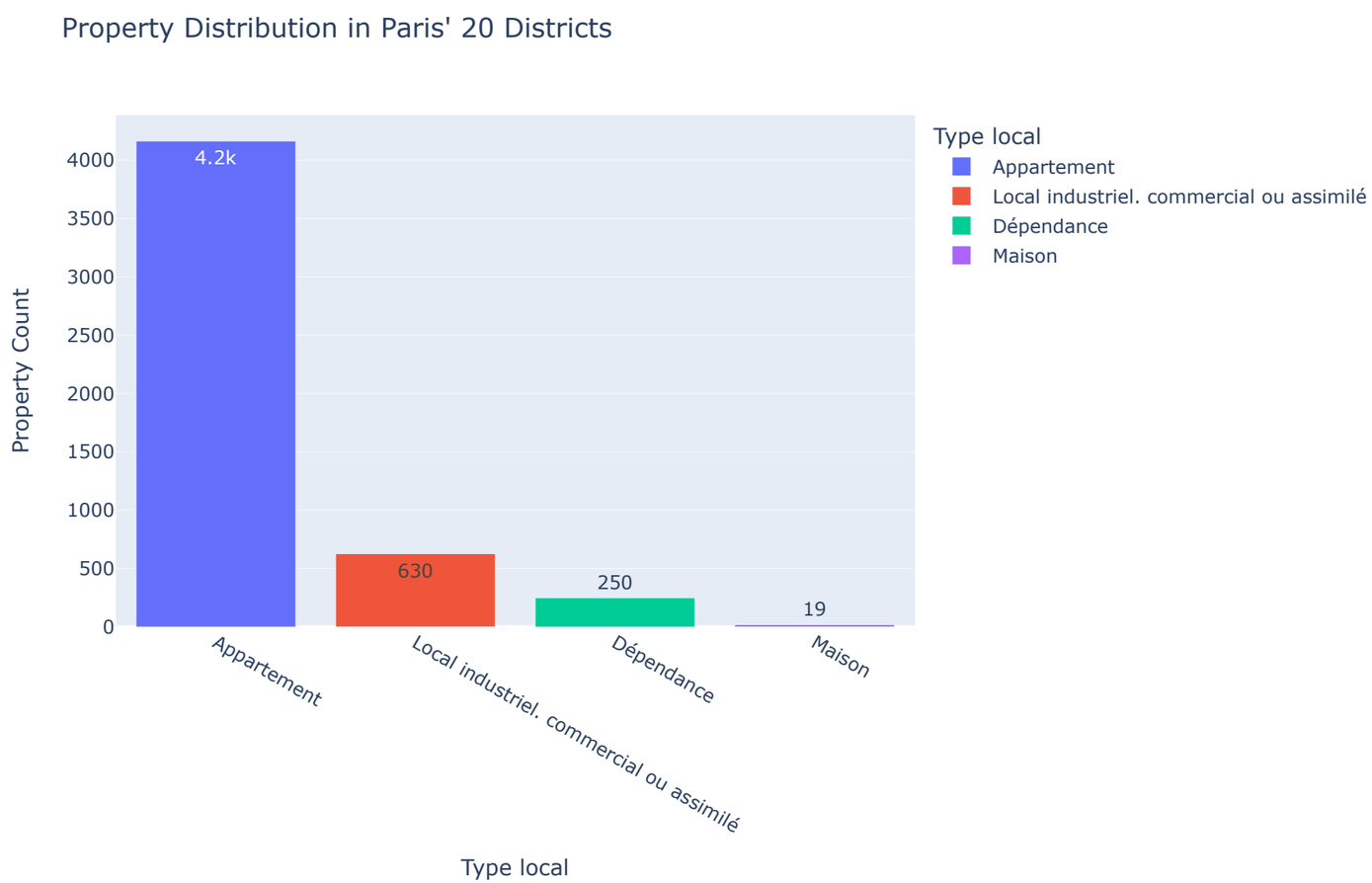
```
    height=600,
)
fig.show()

tmp_df = (
    gdf.groupby(["Commune", "Type local"])
    .agg(property_value=("Valeur fonciere", "mean"))
    .sort_values("property_value", ascending=False)
    .reset_index()
)

fig = px.bar(
    tmp_df,
    x="Commune",
    y="property_value",
    title="Mean Property Value in Paris' 20 Districts",
    labels={"property_value": "Mean Property Value"},
    text_auto=".2s",
    color="Type local",
    barmode="stack",
    width=900,
    height=600,
)
fig.show()

tmp_df = (
    gdf.groupby(["Commune", "Type local"])
    .agg(property_value=("price_per_m2", "mean"))
    .sort_values("property_value", ascending=False)
    .reset_index()
)

fig = px.bar(
    tmp_df,
    x="Commune",
    y="property_value",
    title="Mean Property Value per M^2 in Paris' 20 Districts",
    labels={"property_value": "Mean Property Value per M^2"},
    text_auto=".2s",
    color="Type local",
    barmode="stack",
    width=900,
    height=600,
)
fig.show()

tmp_df = (
    gdf.groupby(["Type local"])
    .agg(property_count=("Type local", "count"))
    .sort_values("property_count", ascending=False)
    .reset_index()
)

fig = px.bar(
    tmp_df,
    x="Type local",
    y="property_count",
    title="Property Distribution in Paris' 20 Districts",
    labels={"property_count": "Property Count"},
    text_auto=".2s",
```

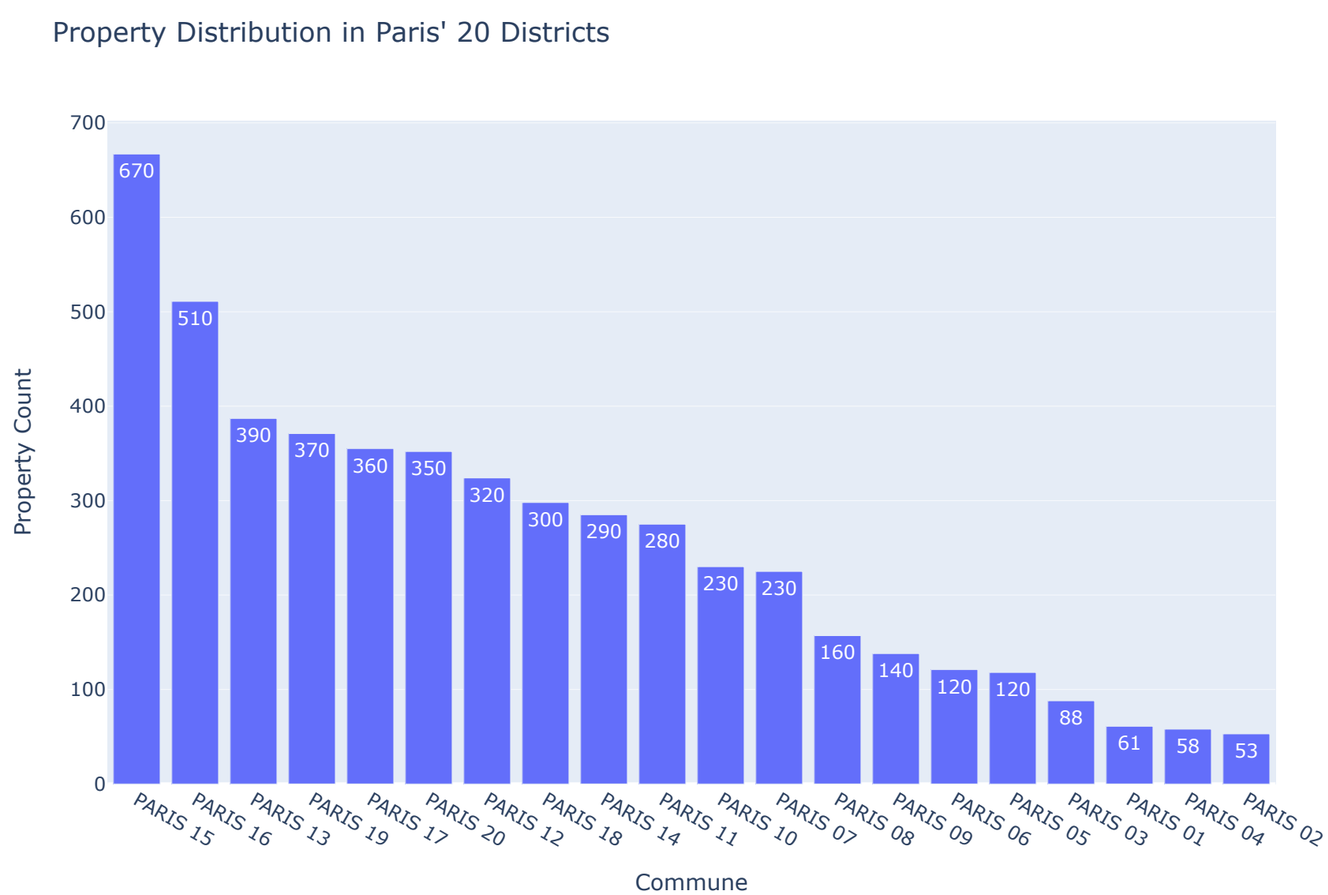
```python
    color="Type local",
    width=900,
    height=600,
)
fig.show()

tmp_df = (
    gdf.groupby(["Commune"])
    .agg(property_count=("Commune", "count"))
    .sort_values("property_count", ascending=False)
    .reset_index()
)
fig = px.bar(
    tmp_df,
    x="Commune",
    y="property_count",
    title="Property Distribution in Paris' 20 Districts",
    labels={"property_count": "Property Count"},
    text_auto=".2s",
    width=900,
    height=600,
)
fig.show()
tmp_df = (
    gdf.groupby(["Commune", "Type local"])
    .agg(property_count=("Type local", "count"))
    .sort_values("property_count", ascending=False)
    .reset_index()
)

fig = px.bar(
    tmp_df,
    x="Commune",
    y="property_count",
    title="Property Distribution in Paris' 20 Districts",
    labels={"property_count": "Property Count"},
    text_auto=".2s",
    color="Type local",
    barmode="stack",
    width=900,
    height=600,
)
fig.show()
```

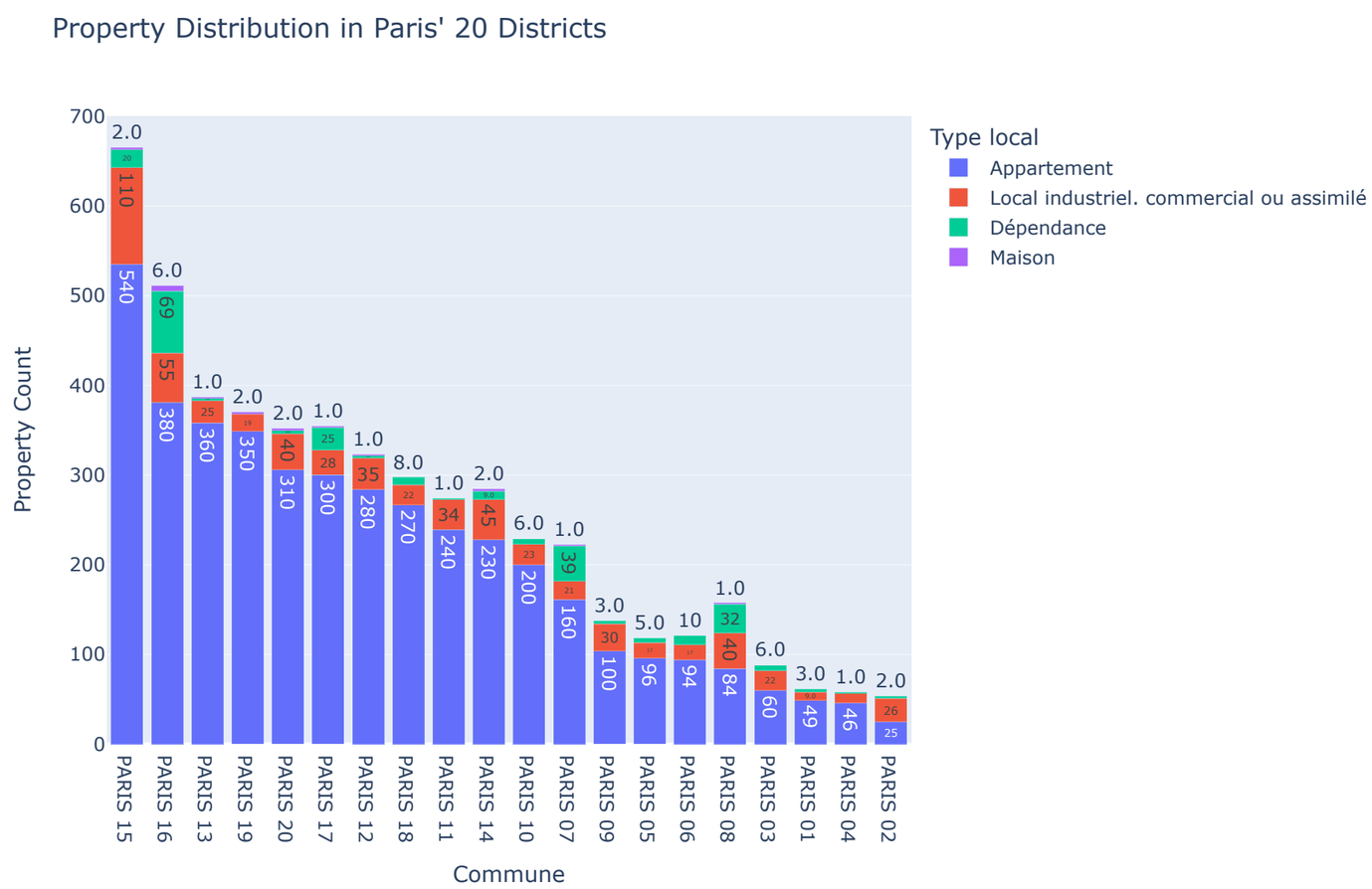

Mean Property Value in Paris' 20 Districts
Mean Property Value
16M
14M
12M
10M
8M
6M
4M
2M
0
16M
5.0M
4.5M
4.2M
3.4M
2.4M
2.3M
1.5M
1.4M
1.3M
1.2M
1.1M
930k
890k
860k
840k
830k
700k
550k
510k
PARIS 02
PARIS 06
PARIS 07
PARIS 08
PARIS 12
PARIS 03
PARIS 16
PARIS 15
PARIS 01
PARIS 09
PARIS 20
PARIS 05
PARIS 17
PARIS 13
PARIS 14
PARIS 04
PARIS 18
PARIS 11
PARIS 10
PARIS 19
Commune
Commune
PARIS 02
PARIS 06
PARIS 07
PARIS 08
PARIS 12
PARIS 03
PARIS 16
PARIS 15
PARIS 01
PARIS 09
PARIS 20
PARIS 05
PARIS 17
PARIS 13
PARIS 14
PARIS 04
PARIS 18
PARIS 11
PARIS 10
PARIS 19


Mean Property Value in Paris' 20 Districts
Mean Property Value
0
20M
40M
60M
80M
100M
Commune
PARIS 07
PARIS 08
PARIS 16
PARIS 20
PARIS 15
PARIS 14
PARIS 12
PARIS 17
PARIS 19
PARIS 13
PARIS 02
PARIS 06
PARIS 03
PARIS 05
PARIS 18
PARIS 09
PARIS 11
PARIS 04
PARIS 01
PARIS 10
1.9M
84M
18M
2.6M
22M
9.5M
1.2M
9.2M
10M
1.1M
4.7M
3.7M
700k
5.7M
590k
3.6M
770k
500k
490k
6.5M
590k
19M
31M
4.7M
9.8M
1.1M
6.7M
710k
530k
1.0M
580k
750k
1.4M
510k
Type local
Maison
Local industriel. commercial ou assimilé
Dépendance
Appartement


Mean Property Value per M^2 in Paris' 20 Districts
Mean Property Value per M^2
250k
200k
150k
100k
50k
0
Commune
PARIS 03
PARIS 02
PARIS 14
PARIS 12
PARIS 13
PARIS 15
PARIS 16
PARIS 20
PARIS 08
PARIS 19
PARIS 06
PARIS 10
PARIS 05
PARIS 04
PARIS 11
PARIS 09
PARIS 07
PARIS 17
PARIS 18
PARIS 01
Type local
Local industriel. commercial ou assimilé
Dépendance
Maison
Appartement


Property Distribution in Paris' 20 Districts
Property Count
4000
3500
3000
2500
2000
1500
1000
500
0
4.2k
630
250
19
Appartement
Local industriel. commercial ou assimilé
Dépendance
Maison
Type local
Type local
Appartement
Local industriel. commercial ou assimilé
Dépendance
Maison


Property Distribution in Paris' 20 Districts
Property Count
700
600
500
400
300
200
100
0
670
510
390
370
360
350
320
300
290
280
230
230
160
140
120
120
88
61
58
53
PARIS 15
PARIS 16
PARIS 13
PARIS 19
PARIS 17
PARIS 20
PARIS 12
PARIS 18
PARIS 14
PARIS 11
PARIS 10
PARIS 07
PARIS 08
PARIS 09
PARIS 06
PARIS 05
PARIS 03
PARIS 01
PARIS 04
PARIS 02
Commune



In [ ]:

```
cols = ["lat", "lon", "commune", "price_per_m2"]

df_mod = gdf[cols].copy()
df_x = df_mod.iloc[:, :-1]
encoder = ce.BinaryEncoder(cols=["commune"], return_df=True)
data_encoded = encoder.fit_transform(df_x)
data_encoded["cat"] = df_x["commune"]
df_encod_dict = data_encoded.iloc[:, 2:].drop_duplicates()
df_y = df_mod["price_per_m2"]

# split the data into train and test (75/25) split and used stratified sampling to ensure a fair representation of each district within the train set.

X_train, X_test, y_train, y_test = train_test_split(
    data_encoded.iloc[:, :-1],
    df_y,
    random_state=0,
    stratify=df_x["commune"].astype(str),
    test_size=0.25,
)

X_train_sc = X_train
```

```python
X_test_sc = X_test

# I set seed for reproducibility sake and performed a grid search for the best hyperparameters for our KNN model.
# frankly the model's performance is poor based R^2 value which is less than 0.1 for the test set.
#
# Clearly the location features alone are not enough to determine the price per square meter. I considered features like number of rooms and property type however those only degraded the model further
#
# Ways to improve it, I believe the location data maybe to granular. I think given more time, and further data wrangling I could aggregate the data to a district level and use district as my location
#
# There could be presents of outliers that is degrading the model, for example district 2, district with the largest mean property, has a mean property value larger than the next 3 districts. Yet it o
# overall location alone is not enough to correctly determine the price per m2.

seed(123)
knnreg = KNeighborsRegressor()
k = np.arange(1, 51)
gr_val = {"n_neighbors": k, "weights": ["uniform", "distance"]}
gr_reg_acc = GridSearchCV(knnreg, param_grid=gr_val, cv=5)
gr_reg_acc.fit(X_train_sc, y_train)

print(f"Grid best parameter (max. accuracy): {gr_reg_acc.best_params_}")
print(f"Grid best score (accuracy): {gr_reg_acc.best_score_}")
print(f"Training R-Squared score: {gr_reg_acc.score(X_train_sc, y_train)}")
print(f"R-squared test score: {gr_reg_acc.score(X_test_sc, y_test)}")
```

```
Grid best parameter (max. accuracy): {'n_neighbors': 5, 'weights': 'uniform'}
Grid best score (accuracy): 0.08246355540760911
Training R-Squared score: 0.19785331576758547
R-squared test score: 0.0726905808614946
```

In [ ]:

```python
def price_per_m2(lat, lon, commune=None):
    temp = pd.DataFrame({"lat": lat, "lon": lon, "commune": commune}, index=[0])
    if commune == None:
        for k, v in mul_pl_lst.items():
            if v.contains(Point(lat, lon)).values[0] == True:
                temp["commune"] = int(k)
                break
    if temp["commune"].values[0] == None:
        return "The location provided is not in Paris."
    temp[df_encod_dict.iloc[:, :-1].columns] = (
        df_encod_dict.iloc[:, :-1]
        .loc[df_encod_dict.cat == temp["commune"].values[0]]
        .values
    )
    temp.drop(columns=["commune"], inplace=True)
    features = [[lat, lon, commune]]
    print(
        f"Based on the coordinates {features[0]}, the approximate price per square meter in that area is:  {gr_reg_acc.predict(temp)}"
    )


price_per_m2(652455.920961, 6.862873e06)
```

```
Based on the coordinates [652455.920961, 6862873.0, None], the approximate price per square meter in that area is:  [5916.64000895]
```